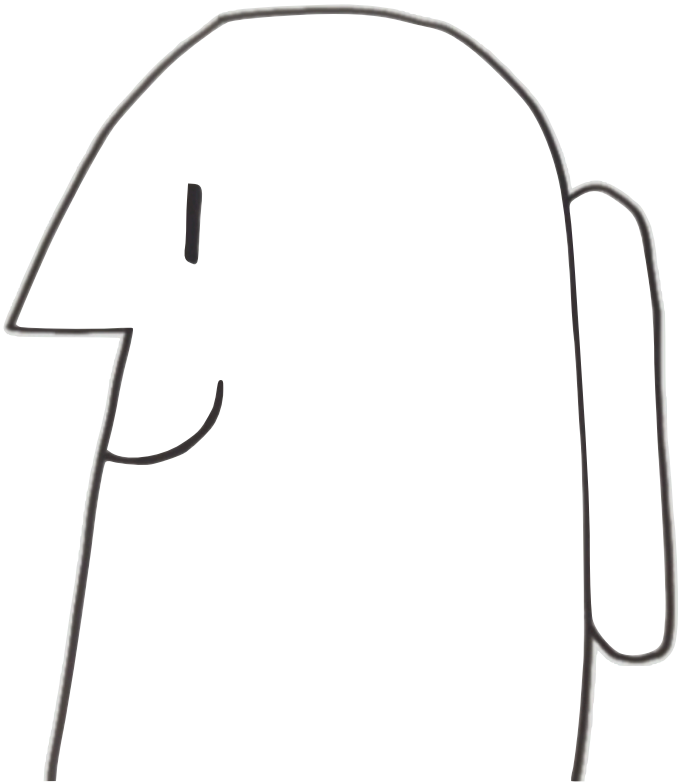
Teapotmon Operation Manual



By Teapot-dev

Teapotmon — минимальный монитор реального режима (real-mode monitor) работающий без операционной системы. Загружается BIOS с загрузочного сектора предоставляет командную строку а также инструменты для ввода и исполнения машинного кода hex-редактор и запуск программ.

Он не является операционной системой. У него нет планировщика нет файловой системы нет защиты памяти. Это слой поверх BIOS прерываний

всё что он умеет читать клавиатуру и печатать на экран

Требования

NASM

QEMU

Сборка

nasm -f bin mon.asm -o mon.bin

Запуск

qemu-system-i386 -fda mon.bin

Список команд

Help выводит список доступных команд

rhc (Run Hex Code) ввод hex-байтов

run запуск кода

Как надо ввести последовательность машинных инструкций для архитектуры x86

text

/ rhc

hex> B4 0E B0 41 CD 10 C3

/ run

A

Ограничения и известные баги

510 байт максимум один загрузочный сектор

Буфер ввода команд — 64 символа

Нет истории команд

Программа запущенная через run может уронить систему если в ней нет инструкции ret (байт C3) Код рассчитан на архитектуру x86, hex-коды от других процессоров (например, 6502) выполняться не будут

```
 1                              [bits 16]
 2                              [org 0x7C00]
 3
 4                              start:
 5 00000000 FA                      cli
 6 00000001 31C0                    xor ax, ax
 7 00000003 8ED8                    mov ds, ax
 8 00000005 8EC0                    mov es, ax
 9 00000007 8ED0                    mov ss, ax
10 00000009 BC007C                      mov sp, 0x7C00
11 0000000C FB                      sti
12
13 0000000D EB00                    jmp main
14
15                              main:
16 0000000F BE[1401]                mov si, prompt
17 00000012 E85400                  call puts
18
19 00000015 BF[5701]                mov di, buffer
20 00000018 E85B00                   call readline
21
22 0000001B BE[1701]                mov si, crlf
23 0000001E E84800                   call puts
24
25 00000021 BE[5701]                mov si, buffer
26 00000024 BF[1A01]                 mov di, cmd_help
27 00000027 E87F00                  call strcmp
28 0000002A 741E                    jz .do_help
29
30 0000002C BE[5701]                  mov si, buffer
31 0000002F BF[1F01]                mov di, cmd_rhc
32 00000032 E87400                   call strcmp
33 00000035 741B                    jz .do_rhc
34
35 00000037 BE[5701]                 mov si, buffer
36 0000003A BF[2301]                  mov di, cmd_run
37 0000003D E86900                   call strcmp
38 00000040 7415                    jz .do_run
39
40 00000042 BE[2701]                  mov si, msg_unknown
41 00000045 E82100                  call puts
42 00000048 EBC5                    jmp main
43
44                              .do_help:
45 0000004A BE[3601]                  mov si, msg_help
46 0000004D E81900                  call puts
47 00000050 EBBD                    jmp main
48
49                              .do_rhc:
50 00000052 E86600                   call rhc
51 00000055 EBB8                    jmp main
52
53                              .do_run:
```

```
 54 00000057 E8B000                call run
 55 0000005A EBB3                  jmp main
 56
 57                          hang:
 58 0000005C F4                    hlt
 59 0000005D EBFD                  jmp hang
 60
 61                          putc:
 62 0000005F B40E                  mov ah, 0x0E
 63 00000061 CD10                  int 0x10
 64 00000063 C3                    ret
 65
 66                          getc:
 67 00000064 B400                  mov ah, 0x00
 68 00000066 CD16                  int 0x16
 69 00000068 C3                    ret
 70
 71                          puts:
 72 00000069 8A04                  mov al, [si]
 73 0000006B 84C0                  test al, al
 74 0000006D 7406                  jz .done
 75 0000006F E8EDFF                call putc
 76 00000072 46                    inc si
 77 00000073 EBF4                  jmp puts
 78                          .done:
 79 00000075 C3                    ret
 80
 81                          readline:
 82                          .loop:
 83 00000076 E8EBFF                call getc
 84 00000079 3C0D                  cmp al, 0x0D
 85 0000007B 7428                  je .enter
 86 0000007D 3C08                  cmp al, 0x08
 87 0000007F 740C                  je .backspace
 88 00000081 3C20                  cmp al, 0x20
 89 00000083 72F1                  jb .loop
 90 00000085 8805                  mov [di], al
 91 00000087 47                    inc di
 92 00000088 E8D4FF                call putc
 93 0000008B EBE9                  jmp .loop
 94
 95                          .backspace:
 96 0000008D 81FF[5701]            cmp di, buffer
 97 00000091 76E3                  jbe .loop
 98 00000093 4F                    dec di
 99 00000094 B008                  mov al, 0x08
100 00000096 E8C6FF                call putc
101 00000099 B020                  mov al, ' '
102 0000009B E8C1FF                call putc
103 0000009E B008                  mov al, 0x08
104 000000A0 E8BCFF                call putc
105 000000A3 EBD1                  jmp .loop
106
107                          .enter:
```

```
108 000000A5 C60500                mov byte [di], 0
109 000000A8 C3                    ret
110
111                            strcmp:
112                            .loop:
113 000000A9 8A04                  mov al, [si]
114 000000AB 3A05                  cmp al, [di]
115 000000AD 7509                  jne .not_equal
116 000000AF 84C0                  test al, al
117 000000B1 7404                  jz .equal
118 000000B3 46                    inc si
119 000000B4 47                    inc di
120 000000B5 EBF2                  jmp .loop
121                            .equal:
122 000000B7 C3                    ret
123                            .not_equal:
124 000000B8 0C01                  or al, 1
125 000000BA C3                    ret
126
127                            rhc:
128 000000BB BE[5101]              mov si, msg_hex
129 000000BE E8A8FF                call puts
130 000000C1 BF0090                mov di, 0x9000
131
132                            .r_loop:
133 000000C4 E89DFF                call getc
134 000000C7 3C0D                  cmp al, 0x0D
135 000000C9 742C                  je .r_done
136 000000CB 3C20                  cmp al, ' '
137 000000CD 74F5                  je .r_loop
138
139 000000CF 50                    push ax
140 000000D0 E88CFF                call putc
141 000000D3 58                    pop ax
142
143 000000D4 E82700                call hex_to_val
144 000000D7 88C1                  mov cl, al
145 000000D9 C0E104                shl cl, 4
146
147                            .r_second:
148 000000DC E885FF                call getc
149 000000DF 3C20                  cmp al, ' '
150 000000E1 74F9                  je .r_second
151
152 000000E3 50                    push ax
153 000000E4 E878FF                call putc
154 000000E7 58                    pop ax
155
156 000000E8 E81300                call hex_to_val
157 000000EB 08C1                  or cl, al
158
159 000000ED 880D                  mov [di], cl
160 000000EF 47                    inc di
161 000000F0 B020                  mov al, ' '
```

```
162 000000F2 E86AFF                call putc
163 000000F5 EBCD                  jmp .r_loop
164
165                        .r_done:
166 000000F7 BE[1701]              mov si, crlf
167 000000FA E86CFF                call puts
168 000000FD C3                    ret
169
170                        hex_to_val:
171 000000FE 3C39                  cmp al, '9'
172 00000100 7605                  jbe .digit
173 00000102 24DF                  and al, 0xDF
174 00000104 2C37                  sub al, 'A' - 10
175 00000106 C3                    ret
176                        .digit:
177 00000107 2C30                  sub al, '0'
178 00000109 C3                    ret
179
180                        run:
181 0000010A E8(0090)              call 0x9000
182 0000010D BE[1701]              mov si, crlf
183 00000110 E856FF                call puts
184 00000113 C3                    ret
185
186 00000114 2F2000            prompt:     db '/ ', 0
187 00000117 0D0A00            crlf:       db 13, 10, 0
188 0000011A 68656C7000          cmd_help:   db 'help', 0
189 0000011F 72686300            cmd_rhc:    db 'rhc', 0
190 00000123 72756E00            cmd_run:    db 'run', 0
191 00000127 73796E746178206572-    msg_unknown: db 'syntax error', 13, 10, 0
191 00000130 726F720D0A00
192 00000136 436F6D6D616E64733A-    msg_help:   db 'Commands: help, rhc, run', 13, 10, 0
192 0000013F 2068656C702C207268-
192 00000148 632C2072756E0D0A00
193 00000151 6865783E2000          msg_hex:    db 'hex> ', 0
194 00000157 00<rep 40h>           buffer:     times 64 db 0
195
196 00000197 00<rep 67h>           times 510-($-$$) db 0
197 000001FE 55AA                  dw 0xAA55
```